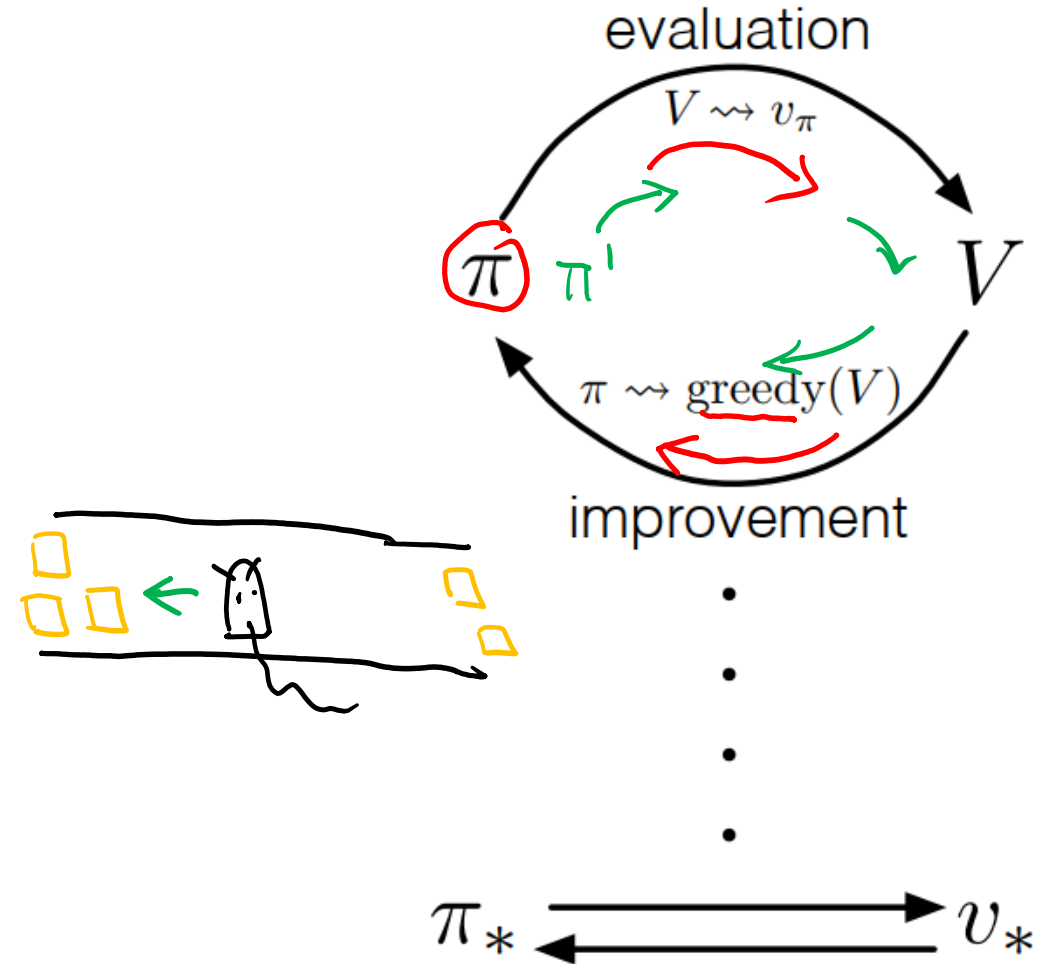


EEE4114F

Temporal Difference Learning

Generalized Policy Iteration

- Need to update policy
- Use policy to evaluate value function
- Use value function to update policy
- Greedy means to choose action that yields the best value
 - $\pi(s) = \operatorname{argmax}_a q_\pi(s, a)$



Exploration and Exploitation

- What happens if we're always greedy?
- Trade-off between exploiting known rewards and exploring other options
 - We may miss out on better states if we don't explore
 - We may end up with less optimal actions if we explore too much
- Soft policies
 - $\pi(a|s) > 0$ for all states and actions
 - ϵ -soft policies: $\pi(a|s) \geq \frac{\epsilon}{|\mathcal{A}(s)|}, \forall a \in \mathcal{A}$
 - ϵ -greedy is ϵ -soft with

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|}, & a = a^* \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & a \neq a^* \end{cases}, \text{ where } \underline{a^*} = \underline{\text{argmax}_a Q(s, a)}$$



Temporal Difference Learning

- $V(S_t) \leftarrow V(S_t) + \alpha [\overbrace{R_{t+1} + \gamma V(S_{t+1})}^{\text{error}} - \underbrace{V(S_t)}_{\text{estimate}}]$
 - α is a learning rate.
 - We could make it a function, say $\alpha(n) = \frac{1}{n}$ where n is the number of visits to the state → bootstrapping
 - Update upon transition to next state.
 - Based on sample updates rather than complete distribution of future states and rewards.
 - Incremental updates (no need to wait until episode is complete).
 - No need to solve simultaneous equations.
 - $TD(0)$, one-step TD.
- $\delta_t \doteq R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$
 - TD error
 - Difference between estimate value of state and new estimate after sampling

Sarsa

- $Q_{\pi}(S_t, A_t) \leftarrow Q_{\pi}(S_t, A_t) + \alpha [R_{t+1} + \gamma Q_{\pi}(S_{t+1}, A_{t+1}) - Q_{\pi}(S_t, A_t)]$
- On-policy control
 - Use same policy for behaviour and estimation

$S_t A_t R_{t+1} S_{t+1} A_{t+1}$
 error
 same policy

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialize S

Choose A from S using policy derived from Q (e.g., ε -greedy)

Loop for each step of episode:

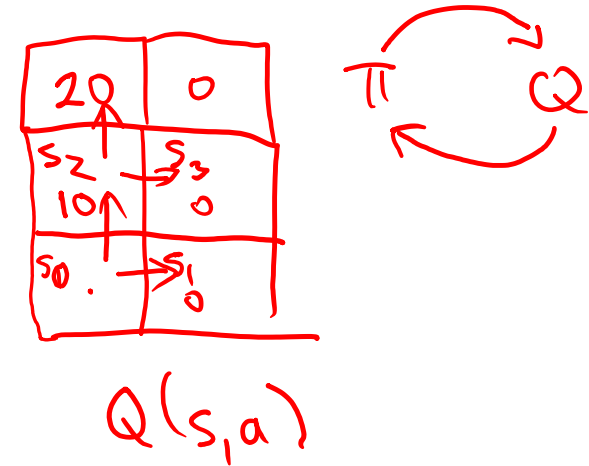
Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

until S is terminal



Q-learning

$$Q_{\pi}(s_{t+1}, A_{t+1})$$

- $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \underbrace{\gamma \max_a Q(S_{t+1}, a)}_{Q_{\pi}(s_{t+1}, A_{t+1})} - Q(S_t, A_t)]$
- Off-policy
 - Uses one policy for behaviour and one for estimation (greedy)

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialize S

Loop for each step of episode:

Choose A from S using policy derived from Q (e.g., ε -greedy)

Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

until S is terminal

